



به نام خدا

تمرین کامپیوتری سوم درس طراحی کامپایلر

بهار ۱۴۰۵

ددلاین: انتهای جمعه ۵ تیر

طراحان: سجاد حسن پور، مهدیس میرزایی

فهرست مطالب

1	فهرست مطالب
2	مقدمه
3	Name Analysis خطاهای
3	1- استفاده از متغیر (شامل ماژول و استراکت) تعریف نشده
4	2- استفاده از متد تعریف نشده
5	3- استفاده از فیلد یا متد private
6	4- تعریف مجدد
7	5- استفاده از متغیر بدون مقدار
8	6- حذف کردن ماژول و استراکت‌های غیر قابل دسترس‌ها از main (امتیازی)
10	Type Analysis خطاهای
10	1- مشکل تایپ در مقداردهی
11	2- غیر منطقی بودن عبارت شرطی
12	3- اشتباه بودن تایپ خروجی متد
13	4- تغییر دادن متغیری که mut نیست
14	5- خطاهای موقع صدا زدن متد

مقدمه

در فاز اول پروژه، تحلیل‌گر لغوی و نحوی (lexer and parser) زبان MOL را پیاده‌سازی کردید. در این فاز از پروژه از شما انتظار می‌رود یک name analyzer برای زبان طراحی کنید. با پیمایش AST، اطلاعات مربوط به توابع، pattern ها و متغیرها را در جدول علائم (Symbol Table) ذخیره کنید. برای این کار لازم است تا متد visit از Visitor interface در NameAnalyzer visitor، برای همه node ها override شود و بررسی‌های لازم برای هر node انجام شود.

بعد از آن باید مرحله‌ی بهینه‌سازی کد را انجام دهید. هدف این مرحله، شناسایی و حذف بخش‌هایی از کد است که بلااستفاده، مرده یا غیر ضروری هستند تا برنامه‌ای کاراتر، خواناتر و ساده‌تر تولید شود. بهینه‌سازی کد نه تنها باعث کاهش حجم برنامه و مصرف کمتر منابع می‌شود، بلکه تحلیل‌های بعدی روی کد (مانند بهینه‌سازی‌های زمان اجرا یا تولید کد ماشین) را نیز بسیار مؤثرتر و ساده‌تر می‌کند. همچنین کد بهینه شده، درک و نگهداری را برای توسعه‌دهندگان بعدی آسان‌تر می‌کند. می‌توانید بخشی از بهینه‌سازی را در NameAnalyzer پیاده‌سازی کرده و بقیه آن را در کلاس(ها) visitor دیگری انجام دهید.

خطاهای Name Analysis

1- استفاده از متغیر (شامل ماژول و استراکت) تعریف نشده

برای استفاده از هر متغیر (شامل ماژول و استراکت یا فیلدهای آنها)، لازم است ابتدا آن متغیر در محدوده (Scope) مربوطه تعریف شده باشد. اگر متغیری قبل از استفاده تعریف نشده باشد، خطای زیر تولید می‌شود:

Line <LineNumber> : <Name> not declared

```
1 module MutabilityTest begin
2
3     int calculateNewValue(int currentValue) begin
4         return currentValue + 10;
5     end
6
7     void main() begin
8         counter = 5;
9         counter = calculateNewValue(counter);
10        output counter;
11    end
12 end
```

در این کد counter تعریف نشده است و باید به خط شماره ۸ و ۹ ارور بدهد.

```
1 Line 8 : counter not declared
2 Line 9 : counter not declared
```

2- استفاده از متد تعریف نشده

اگر متدی در ماژولی تعریف نشده باشد و استفاده شود مشابه خطا قبلی باید ارور دهد.

Line <LineNumber> : <MethodName> not declared

```
1 module MutabilityTest begin
2
3   int calculateNewValue(int currentValue) begin
4     return currentValue + 10;
5   end
6
7   void main() begin
8     mut int counter = 5;
9     counter = calculate(counter);
10    output counter;
11  end
12 end
```

و همانطور که مشخص است calculate تعریف نشده است و باید ارور بدهد.

```
1 Line 9 : calculate not declared
```

3- استفاده از فیلد یا متد private

اگر از یک فیلد یا متدی که پرایوت است استفاده شود باید اروری داده شود که این کار مجاز نبوده است.

Line <LineNumber> : <Name> is private

```
1 module Account begin
2   private int balance;
3   public Account(int initialBalance) begin
4     this.balance = initialBalance;
5   end
6   public int getBalance() begin
7     return this.balance;
8   end
9   public void deposit(int amount) begin
10    if (amount > 0) begin
11      this.balance = this.balance + amount;
12      output this.balance;
13    end
14  end
15 end
16 module Main begin
17   public void main() begin
18     mut Account(100) myAccount;
19     mut int i = 1;
20     mut int nb = myAccount.balance;
21     output myAccount.getBalance();
22     while (i <= 3) begin
23       output i;
24       myAccount.deposit(i * 10);
25       i = i + 1;
26     end
27     output myAccount.getBalance();
28   end
29 end
```

و چون balance یک مقدار private بوده است و نباید از آن استفاده شود پس خط ۲۰ ارور دارد.

```
1 Line 20 : balance is private
```

4- تعریف مجدد

اگر یک متغیر یا فیلد یا متد یا یک ماژول - استراکت دوبار یا بیشتر تعریف شود باید ارور بدهد.

Line <LineNumber> : <Name> already defined

```
1 module Main begin
2     void main() begin
3         mut int a = 10;
4         int b = 5;
5         a = a + b * 2;
6         output a;
7         output (a + b) * 2;
8     end
9     void main() begin
10        mut int a = 100;
11    end
12 end
```

و برای بار دوم main با همان ویژگی‌ها (return type یکسان و آرگومان‌های یکسان) تعریف شده است پس باید ارور دهد.

```
1 Line 9 : main already defined
```

5- استفاده از متغیر بدون مقدار

اگر یک متغیر یا فیلد مقداری نداشته باشد و از مقدار آن استفاده شده باشد (مثل چاپ کردن مقدار آن، استفاده شدن در سمت راست یک تساوی و ...) باید ارور دهد که مقداری برای آن در قبل در نظر نگرفته شده است. دقت کنید اگر پیش از این مقداردهی شده باشد دیگر نباید این ارور چاپ شود.

Line <LineNumber> : <Name> is uninitialized

```
1 module Main begin
2     void main() begin
3         mut int a = 10;
4         int b;
5         a = a + b * 2;
6         output a;
7         output (a + b) * 2;
8     end
9 end
```

و در خط 5 و 7 از مقدار b استفاده می‌شود درحالی‌که هیچ مقداری برای آن در نظر نگرفته شده است.

```
1 Line 5 : b is uninitialized
2 Line 7 : b is uninitialized
```

6- حذف کردن ماژول و استراکت‌های غیر قابل دسترس‌ها از main (امتیازی)

در فرآیند بهینه‌سازی کد، یکی از مهم‌ترین تکنیک‌ها حذف بخش‌هایی از برنامه است که هرگز در جریان واقعی اجرای برنامه مورد استفاده قرار نمی‌گیرند. در این پروژه، مبنای بهینه‌سازی بر اساس تحلیل دسترسی (Reachability) از تابع main تعریف شده است. به این صورت که فقط ماژول‌ها و استراکت‌هایی از برنامه که با شروع پیمایش از main و از طریق زنجیره‌ای از فراخوانی‌های مستقیم یا غیرمستقیم قابل دسترسی باشند، در کد باقی خواهند ماند. به عبارتی ماژول (یا استراکت) A را حذف می‌کنیم و کد مرده در نظر می‌گیریم اگر هیچ ماژولی (یا استراکت) مانند B وجود نداشته باشد که از main بتوان به B رسید و در هیچ قسمتی از B از A استفاده نشده باشد. دقت کنید ممکن است در یک ماژول (یا استراکت) مثل C از A استفاده شده باشد ولی اگر C نیز از main قابل دسترسی نباشد هر دو A و C کد مرده هستند و باید حذف شود. دقت کنید در این فاز ما به بررسی اینکه آیا به این ماژول از لحاظ منطقی می‌توان آن رسید یا نه کاری نداریم. برای مثال اگر در main یک if آمده باشد که شرط آن false باشد ما می‌دانیم این if هیچ‌گاه اجرا نمی‌شود ولی اگر در آن از ماژول D استفاده شده باشد D به عنوان کد مرده در نظر گرفته نمی‌شود. به عبارتی معیار دسترسی از main را به این صورت در نظر بگیرید که آن ماژول یا استراکت آیا در main استفاده شده است؟ سپس بررسی کنید در هر یک از این ماژول‌ها و استراکت‌ها چه ماژول‌ها و استراکت‌های دیگری نیز استفاده شده است و در نهایت به عنوان warning نمایش دهید و در فایل آن قسمت‌ها حذف کنید.

Warning Line <LineNumber> : <Name> is unreachable

```

1 module Math begin
2     public int factorial(int n) begin
3         mut int result = 1;
4         mut int i;
5
6         for (i = 1; i <= n; i = i + 1) begin
7             result = result * i;
8         end
9
10        return result;
11    end
12 end
13
14 module Main begin
15     public void main() begin
16         mut int f;
17         f = 10;
18         output f;
19     end
20 end

```

و ماژول Math از Main هیچ دسترسی‌ای ندارد و حذف می‌شود و ارور به صورت زیر چاپ

```

1 Warning Line 1 : Math is unreachable

```

و فایل به صورت زیر خواهد شد.

```

1 module Main begin
2     public void main() begin
3         mut int f;
4         f = 10;
5         output f;
6     end
7 end

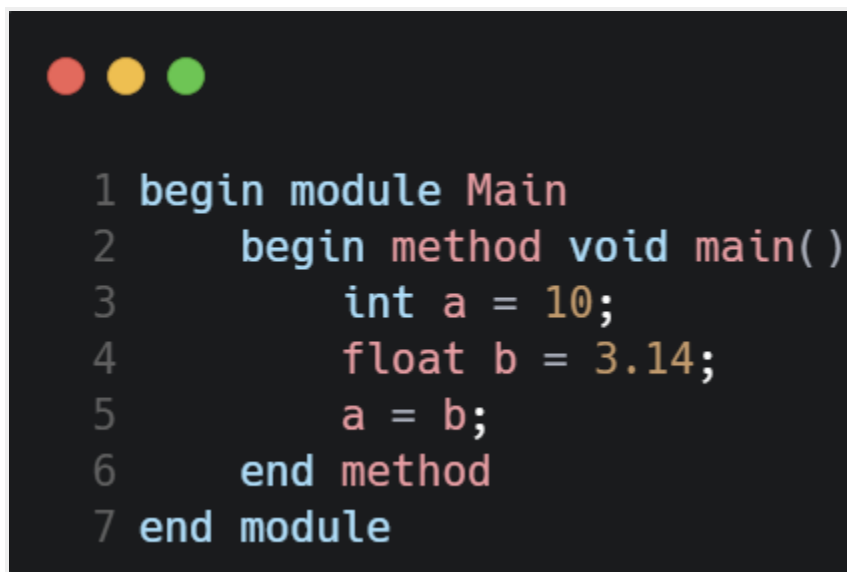
```

خطاهای Type Analysis

1- مشکل تایپ در مقداردهی

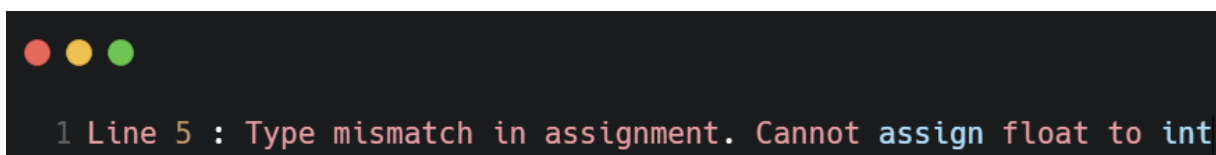
موقع مقداردهی به یک متغیر، باید تایپ عبارت سمت راست با تایپ متغیر سمت چپ همخوانی داشته باشد. در این زبان نمی‌توانید یک تایپ بزرگ‌تر یا متفاوت (مثلاً float یا double) را درون یک متغیر با تایپ دیگر (مثلاً int) بریزید. در صورت وقوع این اتفاق، خطای زیر تولید می‌شود:

Line <LineNumber> : Type mismatch in assignment. Cannot assign <SourceType> to <TargetType>



```
1 begin module Main
2     begin method void main()
3         int a = 10;
4         float b = 3.14;
5         a = b;
6     end method
7 end module
```

در این کد متغیر a از نوع int است، اما تلاش شده تا مقدار متغیر b که float است درون آن ریخته شود، بنابراین در خط ۵ ارور زیر چاپ می‌شود:



```
1 Line 5 : Type mismatch in assignment. Cannot assign float to int
```

2- غیر منطقی بودن عبارت شرطی

در ساختارهای کنترلی مثل if و while، عبارتی که به عنوان شرط داخل پرانتز قرار می‌گیرد حتماً باید از نوع bool باشد. استفاده از انواع عددی (مثل int) به جای شرط مجاز نیست.

Line <LineNumber> : Condition type must be bool

```
1 begin module Main
2     begin method void main()
3         int flag = 1;
4         if (flag) begin
5             int c = 2;
6         end
7     end method
8 end module
```

در خط ۴، متغیر flag که از نوع int است به عنوان شرط if استفاده شده، در حالی که شرطها حتماً باید bool باشند؛ پس باید ارور بدهد:

```
1 Line 4 : Condition type must be bool
```

۳- اشتباه بودن تایپ خروجی متد

مقداری که یک متد return می‌کند، باید دقیقاً با تایپی که در زمان تعریف متد مشخص کرده‌اید، یکی باشد.

Line <LineNumber> : Return type mismatch. Expected <ExpectedType>, got
<ActualType>

```
1 begin module MathUtils
2     begin method int getPi()
3         float pi = 3.14;
4         return pi;
5     end method
6 end module
```

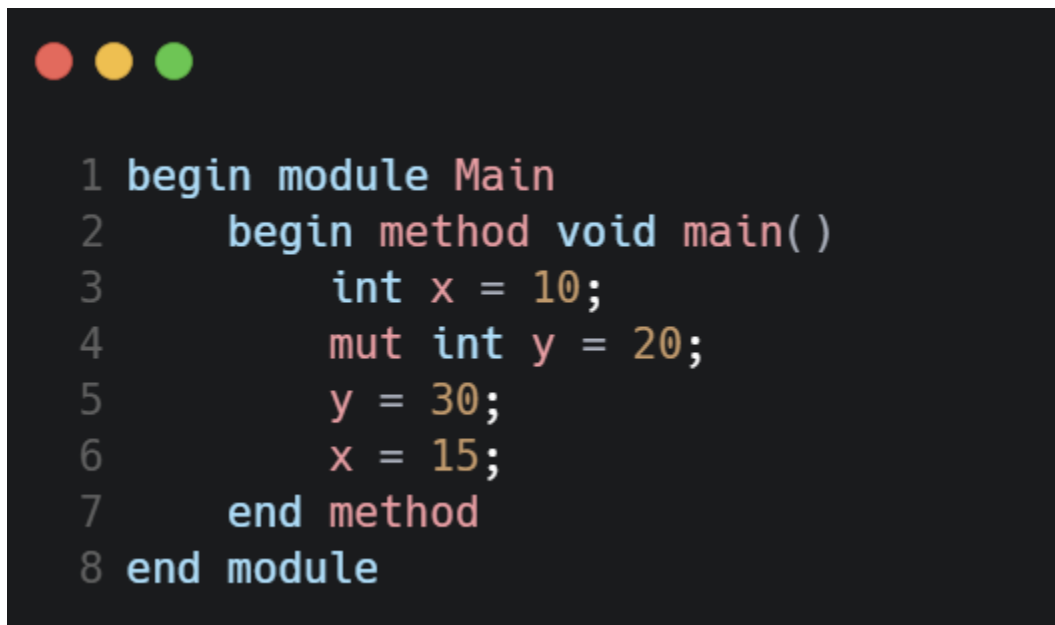
همان‌طور که مشخص است تایپ خروجی متد int تعریف شده، اما در خط ۴ مقدار pi که یک مقدار float است return شده، پس خط ۴ ارور دارد:

```
1 Line 4 : Return type mismatch. Expected int, got float
```

4- تغییر دادن متغیری که mut نیست

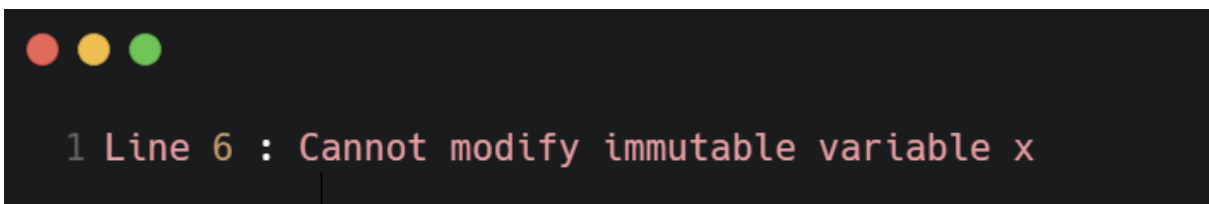
در زبان MOL تمامی متغیرها به صورت پیش فرض غیرقابل تغییر (Immutable) هستند. اگر متغیری موقع تعریف کلمه کلیدی mut را نداشته باشد، برنامه نویس حق ندارد در ادامه برنامه مقدار آن را تغییر دهد (مثلاً در سمت چپ یک تساوی از آن استفاده کند).

Line <LineNumber> : Cannot modify immutable variable <Name>



```
1 begin module Main
2   begin method void main()
3     int x = 10;
4     mut int y = 20;
5     y = 30;
6     x = 15;
7   end method
8 end module
```

متغیر y با کلمه کلیدی mut تعریف شده و تغییر آن در خط ۵ مشکلی ندارد، اما چون x بدون mut تعریف شده است، تغییر مقدار آن در خط ۶ مجاز نیست و ارور می‌دهد:



```
1 Line 6 : Cannot modify immutable variable x
```

5- خطاهای موقع صدا زدن متد

وقتی یک متد را فراخوانی می‌کنید، ورودی‌هایی که به آن می‌دهید (آرگومان‌ها) باید از نظر تعداد و تایپ دقیقاً با تعریف متد یکسان باشند. اگر مشکلی باشد دو حالت خطا داریم:

الف) اشتباه بودن تعداد ورودی‌ها:

Line <LineNumber> : Argument count mismatch for method <MethodName>.

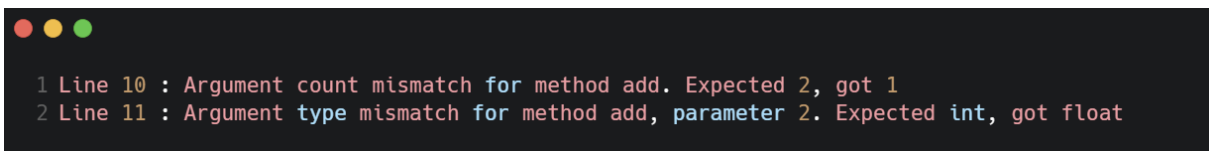
Expected <ExpectedCount>, got <ActualCount>

ب) اشتباه بودن تایپ ورودی‌ها: (در اولین پارامتری که از سمت چپ تطابق نداشت خطا بدهید):

Line <LineNumber> : Argument type mismatch for method <MethodName>,
parameter <ParameterIndex>. Expected <ExpectedType>, got <ActualType>

```
1 begin module Calculator
2     begin method int add(int x, int y)
3         return x + y;
4     end method
5 end module
6
7 begin module Main
8     includes Calculator
9     begin method void main()
10         Calculator.add(5);
11         Calculator.add(5, 2.5);
12     end method
13 end module
```

متد add دو ورودی از نوع int می‌گیرد. در خط ۱۰، فقط یک آرگومان پاس داده شده که ارور تعداد ورودی می‌دهد و در خط ۱۱، آرگومان دوم به جای int، یک مقدار float است که باعث خطای تایپ می‌شود:



```
1 Line 10 : Argument count mismatch for method add. Expected 2, got 1
2 Line 11 : Argument type mismatch for method add, parameter 2. Expected int, got float
```